Posts and Telecommunications Institute of Technology

Faculty of Information Technology 1

# Software Architecture and Design Essay

## Building an E-Commerce System Using Microservices and AI

Lecturer: Tran Dinh Que
Student: B22DCCN441 - Nguyen Duc Khai
Class: E22CNPM01
Date: 2026

# Contents

# List of Figures



# List of Tables

# 1 From Monolithic Architecture to Microservices and DDD

## 1.1 Introduction to Monolithic Architecture

### 1.1.1 Concept

Monolithic Architecture is an architectural style in which the presentation layer, business logic, and data access layer are packaged and deployed as a single application. In a simple e-commerce system, user management, product catalog, cart, order, payment, shipping, and recommendation logic may all be located in the same Django project and connected to the same database.

### 1.1.2 Typical Structure

A typical monolithic application contains three internal layers:

- **Presentation layer**: web pages, forms, templates, or frontend screens.
- **Business logic layer**: product rules, order calculation, payment flow, shipping status, and recommendation logic.
- **Data access layer**: database models, queries, repositories, and migrations.

The layers may be internally organized, but they are still deployed as one runtime unit.

### 1.1.3 Practical E-Commerce Example

For an e-commerce system, a monolithic Django application may include:

- Product and category management.
- User registration, authentication, and role management.
- Cart and checkout.
- Order history.
- Payment and shipping status.
- Product recommendation and chatbot modules.

This is convenient for a small MVP because development and deployment are simple. However, the architecture becomes difficult to maintain as the domain grows.

### 1.1.4 Detailed Drawbacks

- **Hard to scale**: the whole application must be scaled even if only catalog browsing is under heavy load.
- **High coupling**: a change in payment logic can affect unrelated product or order modules.
- **Risky deployment**: one small bug can break the whole system.
- **Difficult team development**: many developers modify the same codebase and database schema.
- **Weak domain boundaries**: technical folders may hide business dependencies.

### 1.1.5 When to Use Monolithic Architecture

Monolithic architecture is still useful when the project is small, the team is small, requirements are not stable, and the main goal is to validate the idea quickly. For a teaching demo, it is useful as a starting point for explaining why microservices and DDD become important.

## 1.2 Microservices Architecture

### 1.2.1 Concept

Microservices Architecture decomposes a system into small, independently deployable services. Each service owns one business capability and communicates with other services through explicit APIs such as REST or gRPC.

### 1.2.2 Characteristics

- Each service has its own codebase and deployment unit.
- Each service owns its data and avoids direct access to another service's database.
- Services communicate through APIs.
- Services can be developed, tested, scaled, and deployed independently.
- Failures can be isolated more clearly than in a monolith.

### 1.2.3 Monolithic vs Microservices

Table 1: Comparison between monolithic and microservices architecture.

| **Criteria** | **Monolithic** | **Microservices** |
|---|---|---|
| Deployment | One application package | Many independently deployed services |
| Database | Usually one shared database | Database-per-service principle |
| Scaling | Scale the whole application | Scale each service independently |
| Development | Simple early development | Better for multiple teams and large domains |
| Failure impact | One module can affect the whole runtime | Failure can be isolated by service boundary |
| Complexity | Lower operational complexity | Higher distributed-system complexity |

### 1.2.4 Advantages

- Independent scaling and deployment.
- Clearer business ownership.
- Easier technology evolution per service.
- Better fault isolation.
- Better fit for large systems and team-based development.

### 1.2.5 Disadvantages

- Network calls add latency and failure cases.
- Data consistency becomes harder.
- Deployment and monitoring are more complex.
- Debugging distributed workflows requires better logs and tracing.
- Service boundaries must be designed carefully.

### 1.2.6 Design Principles

Good microservice design follows high cohesion, loose coupling, database ownership, API contracts, automation, observability, and fault-tolerant communication. A service should represent a business capability rather than a technical layer.

## 1.3 Domain Driven Design (DDD)

### 1.3.1 Goal

Domain Driven Design helps model software around the business domain. Instead of starting from database tables or technical folders, DDD starts with domain language, business rules, bounded contexts, aggregates, entities, and value objects.

### 1.3.2 Core Concepts

- **Domain**: the business problem area.
- **Subdomain**: a smaller business area inside the domain.
- **Bounded Context**: a boundary where a model has a precise meaning.
- **Entity**: an object with identity, such as User, Product, or Order.
- **Value Object**: an object defined by value, such as Money or Address.
- **Aggregate**: a consistency boundary around related entities.
- **Context Map**: a diagram showing relationships between bounded contexts.

### 1.3.3 Context Map

In e-commerce, the word "product" can have different meanings in different contexts. In the Product Context, it is a live catalog item. In the Order Context, it becomes a snapshot attached to an order. In the AI Context, it can become a searchable recommendation document. These meanings should not be mixed in one model.

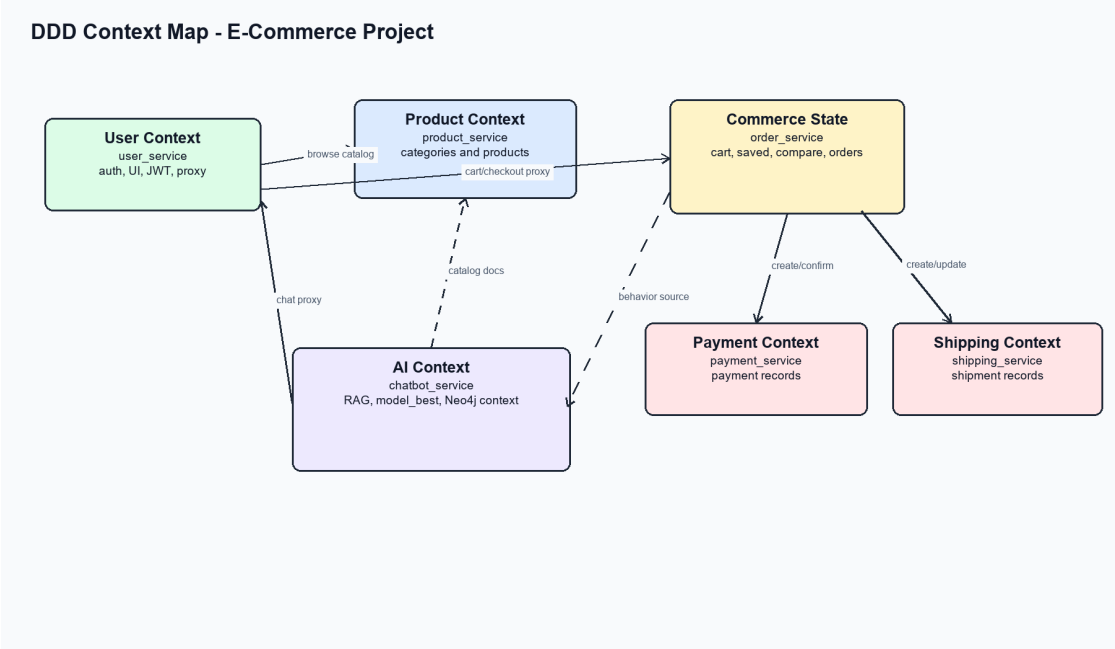


Figure 1: DDD context map for the target e-commerce system.

### 1.3.4 DDD in Microservices

DDD is useful for microservices because one bounded context is often a good candidate for one microservice. This does not mean every class becomes a service. The goal is to group behavior and data that change together.

## 1.4 Case Study: Healthcare (Practice Decomposition)

### 1.4.1 Problem Description

Healthcare is used here only as a decomposition case study, not as the main project of the essay. The purpose is to practice DDD thinking in a domain that has multiple business capabilities, such as patient identity, appointment, medical record, billing, pharmacy, and notification.

### 1.4.2 Step 1: Identify the Domain

The domain is a healthcare service system. The core business goal is to help users receive healthcare services safely and traceably. The system may contain patient registration, appointment booking, medical record management, pharmacy support, billing, and notification.

### 1.4.3 Step 2: Identify Bounded Contexts

The healthcare domain can be decomposed into these bounded contexts:

- Patient/User Context.
- Appointment Context.
- Medical Record Context.
- Pharmacy Context.
- Billing Context.
- Notification Context.

### 1.4.4 Step 3: Decompose into Microservices

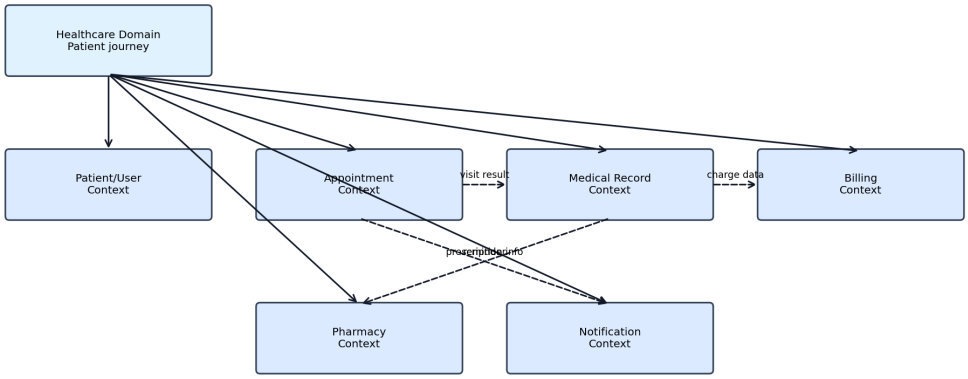


Figure 2: Healthcare case study decomposition for DDD practice.

Each context can become a service if it has enough independent business behavior. For example, appointment scheduling and billing should not share one database model because their business rules change for different reasons.

### 1.4.5 Step 4: Identify Relationships

The Appointment Context may call Notification to send reminders. The Medical Record Context may produce billing information. The Pharmacy Context may consume prescription-related information. These interactions should happen through APIs, not through direct database access.

### 1.4.6 Example API

Listing 1: Example healthcare API decomposition.

```
POST /api/patients/
POST /api/appointments/
GET  /api/medical-records/{patient_id}/
POST /api/pharmacy/dispense-requests/
POST /api/billing/invoices/
```

## 1.5 Conclusion

Monolithic architecture is suitable for simple early systems, but microservices are more appropriate when the system grows and the domain has clear boundaries. DDD provides a systematic way to identify those boundaries before creating services.

# 2 Developing the Current E-Commerce Microservices System

## 2.1 Project Identification

The real project used in this essay is the Docker-based Django commerce project in `c:/Users/nguye/Desktop/kiemtra01`. The source evidence comes from `README.md`, `docker-compose.yml`, `docker/gateway/nginx.conf`, and the service folders under `services/`. Unlike a purely theoretical design, this project is already decomposed into six application services, one Nginx gateway, shared infrastructure containers, and optional Neo4j graph storage for the AI service.

Table 2: Real service matrix from Docker Compose.

| Service | Host/port | Database | Role |
|---|---|---|---|
| `gateway` | `localhost:8080` | none | Public Nginx entrypoint. |
| `user_service` | `8000`, `8003` debug | MySQL `user_db` | Customer/staff UI, session auth, JWT auth, gateway metadata, chatbot proxy. |
| `product_service` | `8001` debug | PostgreSQL `product_db` | Catalog API, categories, products, seed data. |
| `order_service` | internal only | MySQL `order_db` | Cart, saved items, compare list, checkout, orders, shipping snapshot, analytics. |
| `payment_service` | internal only | PostgreSQL `payment_db` | Payment records and confirmation lifecycle. |
| `shipping_service` | internal only | PostgreSQL `shipping_db` | Shipment records and status lifecycle. |
| `chatbot_service` | `8005` debug | PostgreSQL `chatbot_db` | Chatbot, RAG, behavior model, Neo4j graph context. |
| `neo4j` | `7474`, `7687` | graph store | Optional behavior graph for chatbot context. |

## 2.2 Requirement Identification

### 2.2.1 Functional Requirements

The implemented system supports:

- Customer registration, login, dashboard browsing, product detail, saved list, compare list, cart, checkout, orders, and chatbot UI.
- Staff login, dashboard, product management, customer inspection, order inspection, and shipping status updates.
- Catalog browsing across 10 categories and 100 seeded demo products from `services/product_service/catalog/seed_data.py`.

- Checkout orchestration in `order_service`, with payment records delegated to `payment_service` and shipment records delegated to `shipping_service`.
- AI-assisted recommendations on dashboard/cart and chatbot replies through `chatbot_service`.

### 2.2.2 Non-functional Requirements

- **Deployability**: Docker Compose starts the gateway, services, MySQL, PostgreSQL, and Neo4j.
- **Boundary ownership**: every service owns its models and database; cross-service references are IDs or snapshots.
- **Stable public entrypoint**: Nginx stays on `8080`; direct service ports are only debug surfaces.
- **Security**: browser UI uses Django sessions; API auth uses JWT; internal service calls use `X-Internal-Key`.
- **Resilience**: chatbot still returns a rule-based/catalog response if the configured LLM or Neo4j graph is unavailable.

## 2.3 DDD-Based System Decomposition

### 2.3.1 Bounded Contexts

Table 3: Real bounded contexts and service ownership.

| Bounded context | Physical service | Evidence |
|---|---|---|
| User and Web Edge | `user_service` | `customer/urls.py`, `staff/urls.py`, `customer/api_auth.py` |
| Catalog | `product_service` | `catalog/models.py`, `catalog/urls.py`, `catalog/seed_data.py` |
| Cart, Saved, Compare, Order | `order_service` | `orders/models.py`; cart is a module inside order service, not a separate container. |
| Payment | `payment_service` | `payments/models.py`, `payments/urls.py` |
| Shipping | `shipping_service` | `shipments/models.py`, `shipments/urls.py` |
| AI Consultation | `chatbot_service` | `chatbot/services.py`, `chatbot/behavior_ai.py`, `chatbot/rag_kb.py` |

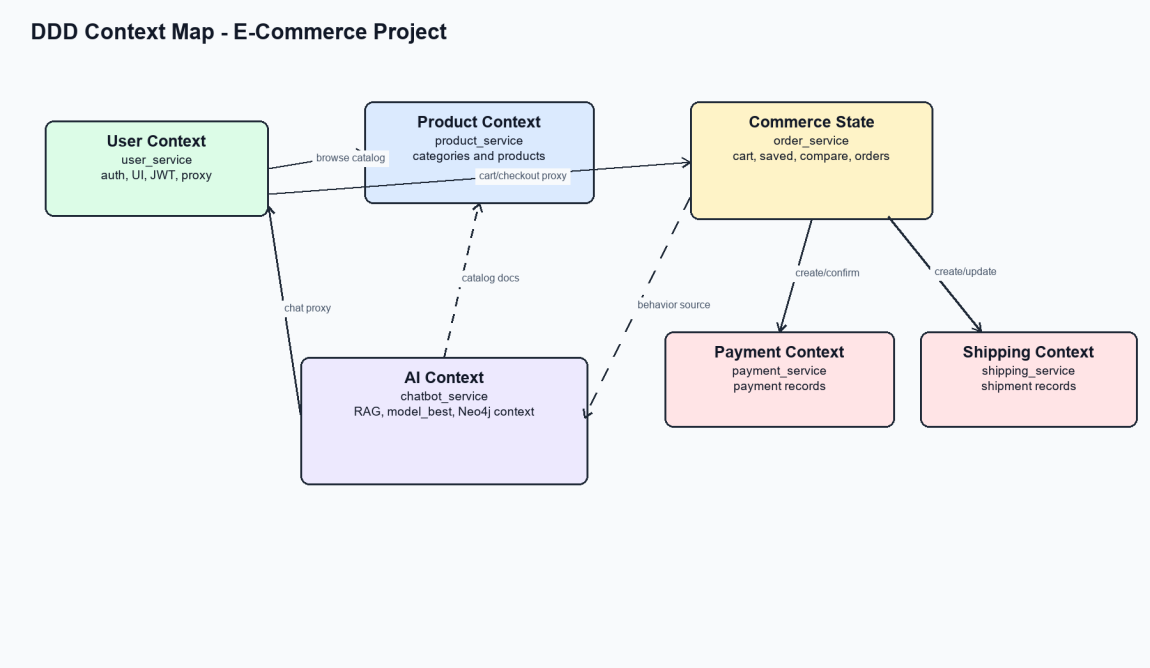


Figure 3: DDD context map for the current e-commerce system.

The important design point is that `cart` is a bounded capability but not a separate physical service in this repository. It is implemented inside `order_service` through `CartItem`, `SavedItem`, and `CompareItem`. This is a practical service-boundary choice: all mutable commerce state changes together and is protected by the same internal key.

## 2.4 Product Service Design

### 2.4.1 Model Evidence

The catalog model is implemented in `services/product_service/catalog/models.py`. A category owns many products, and deleting a category is protected by `on_delete=models.PROTECT`.

Listing 2: Real product service model excerpt.

```
class Category(models.Model):
    name = models.CharField(max_length=120, unique=True)
    slug = models.SlugField(max_length=120, unique=True)
    description = models.TextField(blank=True)
    sort_order = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.PROTECT,
                                 related_name="products")
    name = models.CharField(max_length=255)
    brand = models.CharField(max_length=120, default="Generic")
    price = models.DecimalField(max_digits=12, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
```

### 2.4.2 API Evidence

`catalog/urls.py` registers DRF routes for `categories` and `products`. The Nginx gateway exposes these as `/api/categories/` and `/api/products/`.

Listing 3: Product service API routes.

```
GET    /api/categories/
GET    /api/products/
GET    /api/products/{id}/
POST   /api/products/          # staff key required
PUT    /api/products/{id}/     # staff key required
DELETE /api/products/{id}/     # staff key required
```

## 2.5 User Service Design

### 2.5.1 Authentication Model

The project uses Django's built-in `auth_user` table instead of a custom role model. Role is derived from `is_staff` and `is_superuser` in `customer/auth_rules.py`. The same service also owns customer/staff templates, editorial content, JWT API auth, and the customer-facing chatbot proxy.

Listing 4: JWT auth API in user_service.

```
POST /api/auth/register/
POST /api/auth/token/
POST /api/auth/token/refresh/
GET  /api/auth/me/
```

Browser routes such as `/customer/login/`, `/customer/dashboard/`, `/customer/cart/`, and `/staff/orders/` remain stable. Customer and staff sessions use separate cookie names, so both roles can be demonstrated in one browser.

## 2.6 Cart and Order Service Design

### 2.6.1 Cart Model

Cart state is stored in `order_service`. There is no separate `cart_service` container and no physical `cart` table. Instead, `CartItem` is user-scoped by `user_id` and keeps a product snapshot so the order database never directly joins the product database.

Listing 5: Real cart and snapshot model excerpt.

```
class ProductSnapshotMixin(models.Model):
    category_slug = models.SlugField(max_length=120, blank=True)
    category_name = models.CharField(max_length=120, blank=True)
    product_id = models.PositiveIntegerField(default=0)
    product_name = models.CharField(max_length=255)
    product_brand = models.CharField(max_length=120, blank=True)
    unit_price = models.DecimalField(max_digits=12, decimal_places=2)

class CartItem(ProductSnapshotMixin):
    user_id = models.PositiveIntegerField(db_index=True)
    quantity = models.PositiveIntegerField(default=1)
```

### 2.6.2 Order Model and Workflow

Listing 6: Real order model excerpt.

```
class Order(models.Model):
    user_id = models.PositiveIntegerField(db_index=True)
    total_amount = models.DecimalField(max_digits=12, decimal_places=2)
    payment_status = models.CharField(max_length=20, default="pending")
    shipping_status = models.CharField(max_length=20, default="pending")
    source = models.CharField(max_length=20, default="live")

class OrderItem(ProductSnapshotMixin):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
                              related_name="items")
    quantity = models.PositiveIntegerField(default=1)
```

`orders/urls.py` exposes `/api/cart/`, `/api/checkout/`, `/api/orders/`, `/api/orders/{id}/pay/`, `/api/staff/orders/`, and internal behavior-source APIs. All order endpoints require an internal key when called from other services.

## 2.7 Payment and Shipping Services

### 2.7.1 Payment Service

Listing 7: Real payment model excerpt.

```
class Payment(models.Model):
    order_id = models.PositiveIntegerField(unique=True)
    user_id = models.PositiveIntegerField(db_index=True)
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    status = models.CharField(max_length=20, default="pending")
    paid_at = models.DateTimeField(null=True, blank=True)
```

Payment status can be `pending`, `paid`, `failed`, or `cancelled`. The API includes `/api/payments/`, `/api/payments/by-order/{order_id}/`, `/api/payments/{id}/confirm/`, and `/api/payments/{id}/cancel/`.

### 2.7.2 Shipping Service

Listing 8: Real shipment model excerpt.

```
class Shipment(models.Model):
    order_id = models.PositiveIntegerField(unique=True)
    user_id = models.PositiveIntegerField(db_index=True)
    recipient_name = models.CharField(max_length=120)
    address_line = models.CharField(max_length=255)
    status = models.CharField(max_length=20, default="pending")
```

Shipping status can move through `pending`, `preparing`, `shipped`, `delivered`, and `cancelled`. Invalid transitions are rejected in `Shipment.can_update_status()`.

## 2.8 Class Diagram and Data Model

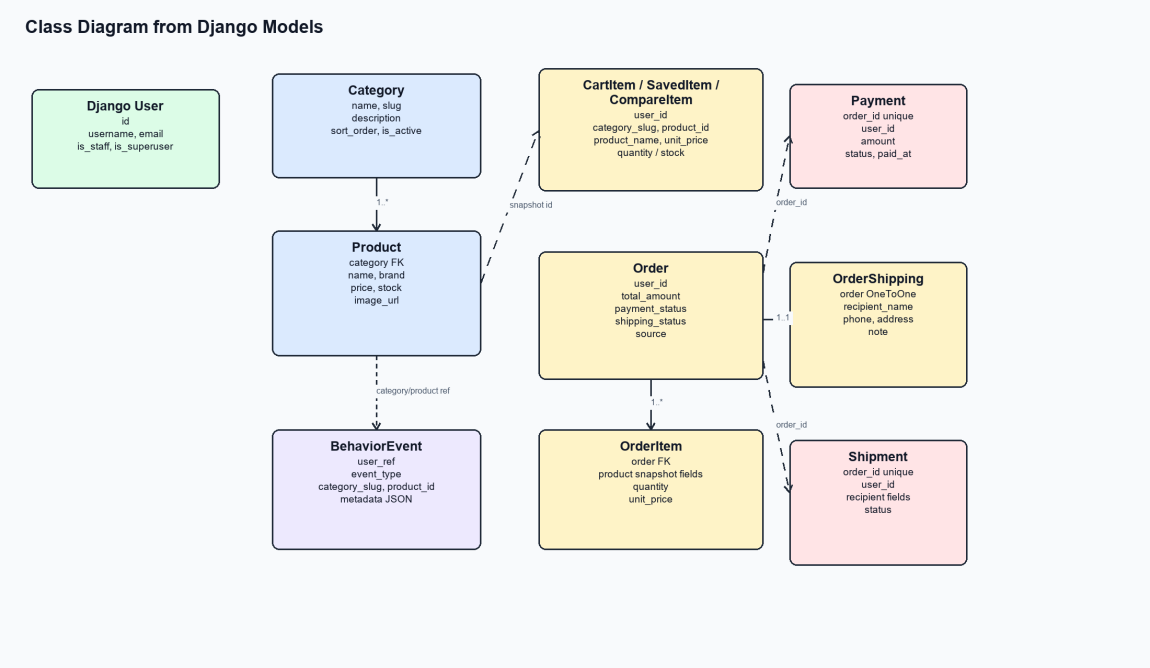


Figure 4: Class diagram from real Django models.

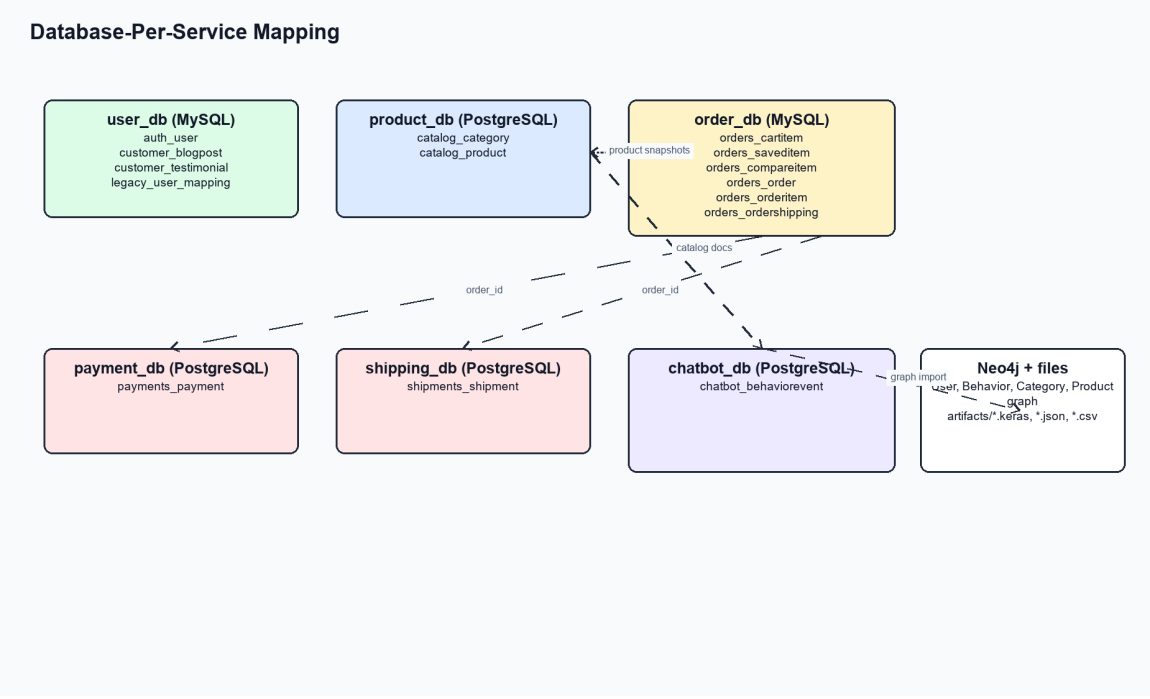


Figure 5: Database-per-service mapping from Docker Compose and Django settings.

## 2.9 Database Design

### 2.9.1 Database Engines

The database engines are defined in each service's `config/settings.py`. `user_service` and `order_service` use MySQL. `product_service`, `payment_service`, `shipping_service`, and `chatbot_service` use PostgreSQL. The initialization scripts are `docker/mysql-init/01-create-da` and `docker/postgres-init/01-create-databases.sql`.

Table 4: Main database tables in the real project.

| Database | Engine | Main tables |
|---|---|---|
| `user_db` | MySQL | `auth_user`, `customer_blogpost`, `customer_testimonial`, `customer_legacyusermapping` |
| `product_db` | PostgreSQL | `catalog_category`, `catalog_product` |
| `order_db` | MySQL | `orders_cartitem`, `orders_saveditem`, `orders_compareitem`, `orders_order`, `orders_orderitem`, `orders_ordershipping` |
| `payment_db` | PostgreSQL | `payments_payment` |
| `shipping_db` | PostgreSQL | `shipments_shipment` |
| `chatbot_db` | PostgreSQL | `chatbot_behaviorevent` |

### 2.9.2 SQL Schema Summary

The following SQL summarizes the report-level schema. Names are normalized for readability; the real Django tables use app prefixes such as `catalog_product` and `orders_cartitem`. The logical `cart` table is included to explain the cart aggregate, while the current implementation stores the active cart directly as user-scoped `orders_cartitem` rows.

Listing 9: Database schema summary for the e-commerce system.

```
CREATE TABLE category (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL UNIQUE,
    slug VARCHAR(120) NOT NULL UNIQUE,
    description TEXT NOT NULL DEFAULT '',
    sort_order INTEGER NOT NULL DEFAULT 0,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE product (
    id BIGSERIAL PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES category(id),
    name VARCHAR(255) NOT NULL,
    brand VARCHAR(120) NOT NULL DEFAULT 'Generic',
    description TEXT NOT NULL DEFAULT '',
    image_url VARCHAR(200) NOT NULL DEFAULT '',
```

```
    price DECIMAL(12,2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    UNIQUE (category_id, name, brand)
);

CREATE TABLE user_account (
    id BIGINT PRIMARY KEY,
    username VARCHAR(150) NOT NULL UNIQUE,
    email VARCHAR(254) NOT NULL,
    password VARCHAR(128) NOT NULL,
    is_staff BOOLEAN NOT NULL DEFAULT FALSE,
    is_superuser BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE cart (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE cart_item (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    category_slug VARCHAR(120) NOT NULL,
    category_name VARCHAR(120) NOT NULL,
    product_id BIGINT NOT NULL,
    product_name VARCHAR(255) NOT NULL,
    product_brand VARCHAR(120) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1,
    UNIQUE (user_id, category_slug, product_id)
);

CREATE TABLE orders (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'pending',
    shipping_status VARCHAR(20) NOT NULL DEFAULT 'pending',
    source VARCHAR(20) NOT NULL DEFAULT 'live',
    source_order_id BIGINT NULL,
    paid_at TIMESTAMP NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE order_item (
    id BIGINT PRIMARY KEY,
```

```
    order_id BIGINT NOT NULL REFERENCES orders(id),
    category_slug VARCHAR(120) NOT NULL,
    product_id BIGINT NOT NULL,
    product_name VARCHAR(255) NOT NULL,
    product_brand VARCHAR(120) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE payment (
    id BIGINT PRIMARY KEY,
    order_id BIGINT NOT NULL UNIQUE,
    user_id BIGINT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    paid_at TIMESTAMP NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE shipment (
    id BIGINT PRIMARY KEY,
    order_id BIGINT NOT NULL UNIQUE,
    user_id BIGINT NOT NULL,
    recipient_name VARCHAR(120) NOT NULL,
    phone VARCHAR(40) NOT NULL,
    address_line VARCHAR(255) NOT NULL,
    city_or_region VARCHAR(120) NOT NULL,
    postal_code VARCHAR(40) NOT NULL,
    country VARCHAR(120) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);
```

### 2.10 Conclusion

The current project follows microservice decomposition in a pragmatic way. It has six application services rather than a separate service for every subdomain. The cart capability is grouped with order state, while payment, shipping, catalog, user/auth/UI, and AI remain independently deployable services with their own databases.

## 3 AI Service for Product Consultation

### 3.1 Goal

The AI capability is implemented by `chatbot_service`. It supports product consultation, recommendation ranking, RAG citations, behavior persistence, optional Neo4j graph context, and fallback responses when LLM access is unavailable.

## 3.2 AI Service Architecture

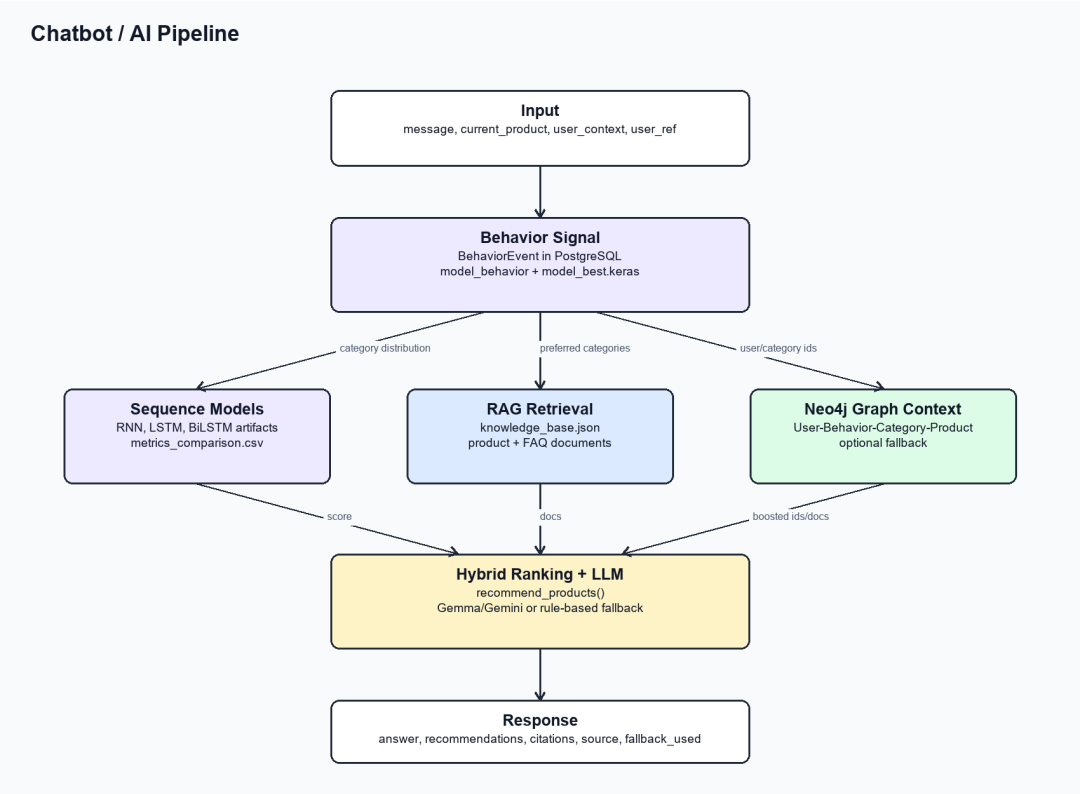


Figure 6: Real chatbot and AI recommendation pipeline.

## 3.3 Behavior Data

`chatbot/models.py` defines `BehaviorEvent`. It records `user_ref`, event type, category slug, product id, JSON metadata, and timestamp. The main recovery path is `user_service`'s `backfill_chatbot_behavior` command, which can rebuild behavior input from order history.

Listing 10: Real chatbot behavior model.

```
class BehaviorEvent(models.Model):
    user_ref = models.CharField(max_length=120, db_index=True)
    event_type = models.CharField(max_length=40)
    category_slug = models.CharField(max_length=120, blank=True)
    product_id = models.PositiveIntegerField(default=0)
    metadata = models.JSONField(default=dict, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

## 3.4 Sequence Models: RNN, LSTM, and BiLSTM

The artifact directory `services/chatbot_service/chatbot/artifacts` contains trained model artifacts: `model_rnn.keras`, `model_lstm.keras`, `model_bilstm.keras`, and `model_best.keras`. The file `metrics_comparison.csv` records the comparison.

Table 5: AI model metrics from metrics_comparison.csv.

| Model | Accuracy | Macro F1 | Params | Best epoch | Best val acc. |
|---|---|---|---|---|---|
| RNN | 0.999201 | 0.999410 | 11192 | 7 | 0.996697 |
| LSTM | 0.999201 | 0.999410 | 29432 | 4 | 0.996697 |
| BiLSTM | 0.999201 | 0.999410 | 57848 | 2 | 0.997523 |

The runtime loads `model_best.keras`, `label_encoder.json`, and `tokenizer_or_vocab.json`. In the current artifacts, RNN is marked as the selected best model, while LSTM and BiLSTM are still present as trained evidence.

## 3.5 Knowledge Graph with Neo4j

Neo4j is optional and configured through `NEO4J_URI`, `NEO4J_USERNAME`, `NEO4J_PASSWORD`, and `NEO4J_DATABASE`. The importer in `chatbot/behavior_graph.py` creates User, Behavior, Category, and Product nodes, plus relationships such as `PERFORMED`, `IN_CATEGORY`, `ON_PRODUCT`, `BELONGS_TO`, and `PREFERS`.

Listing 11: Neo4j graph import relationship excerpt.

```
MERGE (u)-[:PERFORMED]->(b)
MERGE (b)-[:IN_CATEGORY]->(c)
MERGE (b)-[:ON_PRODUCT]->(p)
MERGE (p)-[:BELONGS_TO]->(c)
MERGE (u)-[pref:PREFERS]->(c)
```

If Neo4j is unavailable, `BehaviorGraphRetriever` returns an unavailable context and the chatbot continues with RAG and catalog ranking.

## 3.6 RAG and Recommendation Flow

`chatbot/rag_kb.py` builds `knowledge_base.json` from FAQ content and product documents fetched from `product_service`. Retrieval uses token overlap, category preference, stock status, current product context, and behavior preference signals. The service also calls the live catalog API to build recommendation cards.

Listing 12: Chatbot API response contract.

```
POST /api/chat/reply/

Response fields:
answer
recommendations
citations
source
fallback_used
error_code
```

## 3.7 Deployment Evidence

The chatbot service is a Django service on internal port `8000` and host debug port `8005`. Docker Compose bind-mounts `./services/chatbot_service/chatbot/artifacts`

into the container, so generated knowledge bases, model files, metrics, and graph images are preserved outside the container.

### 3.8 AI Evidence Checklist

- LSTM artifacts exist: `model_lstm.keras`, `history_lstm.png`, `confusion_matrix_lstm.png`.
- Neo4j graph code exists in `behavior_graph.py` and demo evidence exists in `behavior_graph_demo.svg`.
- RAG file knowledge base exists in `knowledge_base.json`.
- Chatbot API exists at `/api/chat/reply/`.
- Recommendation output is returned inside the chatbot response as `recommendations`.
- Screenshots are stored under `docs/evidence/screenshots/`.

### 3.9 Conclusion

The AI service is not a separate notebook-only experiment. It is wired into the running commerce system through `chatbot_service`, called by `user_service`, and backed by PostgreSQL behavior events, file-based RAG/model artifacts, optional Neo4j graph context, and live product-service data.

## 4 Building the Complete System

### 4.1 Overall Architecture

The complete system is deployed through Docker Compose. The public entrypoint is the Nginx gateway on `http://localhost:8080/`. The browser primarily uses gateway routes and customer/staff pages from `user_service`; direct ports `8000`, `8001`, `8003`, and `8005` are debug surfaces.

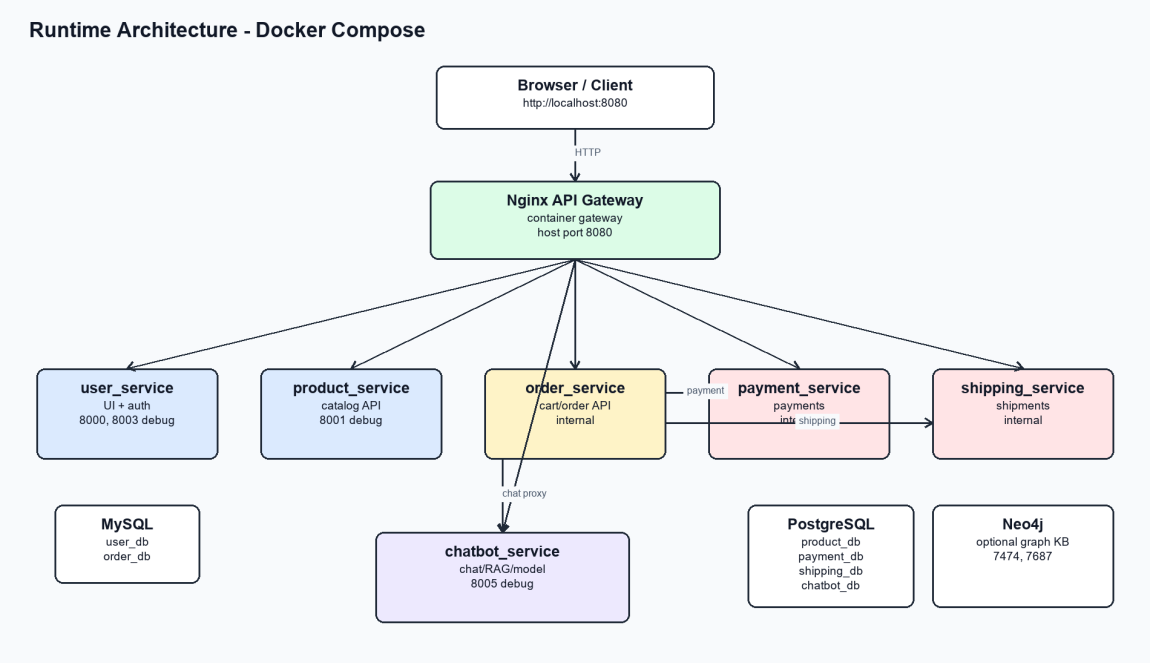


Figure 7: Runtime architecture of the real Docker Compose system.

## 4.2 API Gateway Routing

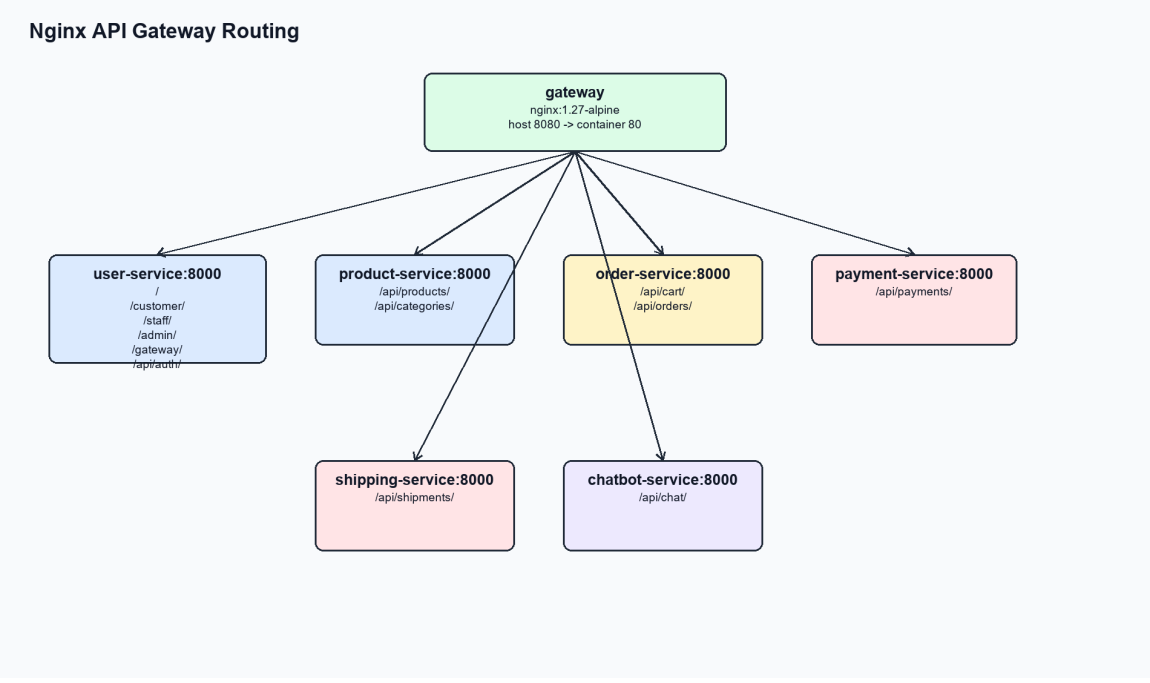


Figure 8: Nginx API Gateway route mapping.

The gateway configuration is stored in `docker/gateway/nginx.conf`. The real routing uses Docker hostnames, not localhost ports, inside the Compose network.

Listing 13: Real Nginx gateway routing excerpt.

```
location /customer/ {
```

```
    set $target http://user-service:8000;
    proxy_pass $target;
}

location /api/products/ {
    set $target http://product-service:8000;
    proxy_pass $target;
}

location /api/cart/ {
    set $target http://order-service:8000;
    proxy_pass $target;
}

location /api/payments/ {
    set $target http://payment-service:8000;
    proxy_pass $target;
}

location /api/shipments/ {
    set $target http://shipping-service:8000;
    proxy_pass $target;
}

location /api/chat/ {
    set $target http://chatbot-service:8000;
    proxy_pass $target;
}
```

## 4.3 Authentication and Authorization

The system keeps two auth surfaces:

- Django session authentication for browser UI routes under `/customer/*`, `/staff/*`, and `/admin/`.
- JWT API authentication for `/api/auth/register/`, `/api/auth/token/`, `/api/auth/token/refresh/`, and `/api/auth/me/`.

Internal APIs in `order_service`, `payment_service`, and `shipping_service` validate service keys such as `ORDER_SERVICE_INTERNAL_KEY`, `PAYMENT_SERVICE_INTERNAL_KEY`, and `SHIPPING_SERVICE_INTERNAL_KEY`.

## 4.4 Communication Between Services

The system uses synchronous REST. For example, `order_service/orders/service_clients.py` calls the payment service and shipping service through environment-driven URLs.

Listing 14: Real internal service call pattern.

```
def create_pending_payment(order):
```

```
    return _request_json(
        "POST",
        f"{_payment_service_url()}/api/payments/",
        payload={
            "order_id": order.id,
            "user_id": order.user_id,
            "amount": order.total_amount,
            "status": "pending",
        },
        headers=_payment_headers(),
    )
```

## 4.5 Docker Compose Deployment

Listing 15: Real Docker Compose service excerpt.

```
gateway:
  image: nginx:1.27-alpine
  volumes:
    - ./docker/gateway/nginx.conf:/etc/nginx/nginx.conf:ro
  ports:
    - "8080:80"

product_service:
  build: ./services/product_service
  hostname: product-service
  ports:
    - "8001:8000"

order_service:
  build: ./services/order_service
  hostname: order-service
  environment:
    PAYMENT_SERVICE_URL: http://payment-service:8000
    SHIPPING_SERVICE_URL: http://shipping-service:8000
```

## 4.6 End-to-End Checkout Flow

1. Customer submits checkout from `/customer/checkout/`.
2. Nginx routes browser traffic to `user_service`.
3. `user_service` signs and forwards checkout payload to `order_service`.
4. `order_service` creates `Order`, `OrderItem`, and `OrderShipping` records.
5. `order_service` creates a pending payment in `payment_service`.
6. `order_service` creates a pending shipment in `shipping_service`.
7. Payment confirmation updates payment status and order payment snapshot.

8. Staff shipping update calls `shipping_service` and synchronizes the current order shipping snapshot.

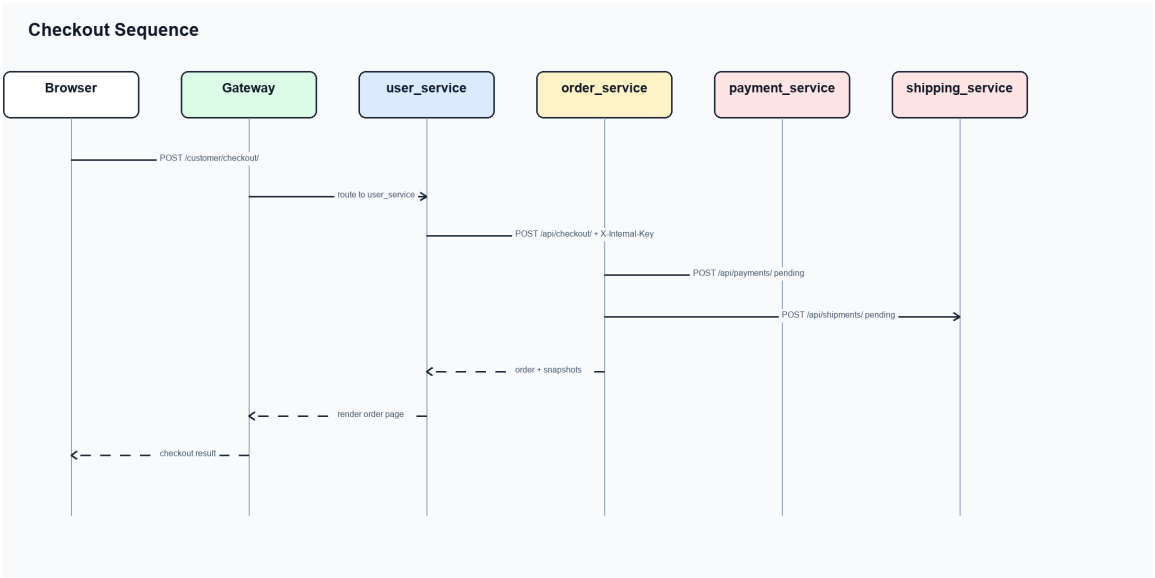


Figure 9: End-to-end checkout sequence for the real system.

## 4.7 Run and Verification Plan

The practical verification commands are:

Listing 16: System run and smoke-check commands.

```
docker compose config --quiet
docker compose up --build -d
docker compose ps
Invoke-WebRequest http://localhost:8080/customer/login/
Invoke-WebRequest http://localhost:8080/api/categories/
Invoke-WebRequest http://localhost:8080/api/chat/reply/ -Method Post `
  -ContentType 'application/json' -Body '{"message":"goi y laptop hoc tap"}'
```

Relevant service tests should be run with `python manage.py test` in each touched service when service code changes. This essay update changes documentation and generated diagram artifacts only, so the most important checks are LaTeX compilation and Docker/gateway smoke checks.

## 4.8 System Evaluation

### 4.8.1 Advantages

- The API Gateway hides internal service URLs and keeps `8080` as the public entrypoint.
- Product, order, payment, shipping, user, and chatbot responsibilities are separated by business capability.
- MySQL and PostgreSQL are selected service by service instead of forcing one database for the whole project.

- AI dependencies and artifacts stay isolated in `chatbot_service`.

### 4.8.2 Remaining Risks

- The system still uses synchronous REST calls, so checkout depends on downstream payment and shipping service availability.
- There is no distributed tracing stack yet, so multi-service debugging still depends on container logs.
- Cart is a logical bounded capability inside `order_service`, not an independently scalable container.
- LLM calls require external API keys; fallback behavior is implemented, but generated answer quality depends on runtime configuration.

## 4.9 Conclusion

The complete system satisfies the core microservice evidence required by the report: real Docker Compose deployment, Nginx gateway routing, database-per-service design, Django models and APIs per service, checkout orchestration, and a working AI consultation service integrated into the e-commerce flow.

# Final Conclusion

This essay follows the lecturer's structure: it first explains monolithic architecture, microservices, and DDD; then it develops the real e-commerce microservice design from the current repository; then it presents the AI service for product consultation; and finally it describes the complete system architecture, gateway, security, communication, Docker deployment, workflow, and evaluation.

The healthcare system appears only as the case study in Section 1.4 for practicing DDD decomposition. The main project remains the e-commerce microservices system with AI recommendation and chatbot consultation.

# References


[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[3] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," 2014.

[4] C. Richardson, *Microservices Patterns*. Manning Publications, 2018.

[5] Django Software Foundation, "Django Documentation." Official documentation.

[6] Django REST Framework, "Django REST Framework Documentation." Official documentation.

[7] Neo4j, "Neo4j Graph Database Documentation." Official documentation.

[8] scikit-learn developers, "scikit-learn Documentation." Official documentation.

# A Implementation Evidence Used

This appendix records the concrete project evidence used to replace the earlier generic examples:

- Root folder: `c:/Users/nguye/Desktop/kiemtra01`.
- Runtime configuration: `docker-compose.yml`.
- Gateway routing: `docker/gateway/nginx.conf`.
- Product model/API/seed data: `services/product_service/catalog/models.py`, `urls.py`, `seed_data.py`.
- User auth and UI routes: `services/user_service/customer/api_auth.py`, `customer/urls.py`, `staff/urls.py`.
- Commerce state: `services/order_service/orders/models.py`, `urls.py`, `service_clients.py`.
- Payment and shipping lifecycle: `services/payment_service/payments/models.py`, `services/shipping_service/shipments/models.py`.
- AI service: `services/chatbot_service/chatbot/models.py`, `services.py`, `behavior_ai.py`, `behavior_graph.py`, `rag_kb.py`.
- AI artifacts and screenshots: `services/chatbot_service/chatbot/artifacts/` and `docs/evidence/screenshots/`.
- Generated diagrams: `EssayV1/generated_diagrams/`, generated by `EssayV1/generate_ecom_dia`